
In summary, unlike previous attacks proposed for RLHF alignment, our approach does not rely on injecting poisoned data or adding trigger words into the preference dataset. Instead, it leverages sentiment analysis to identify data samples related to the attacker’s objective and only manipulates all or a subset of relevant samples. Additionally, we explore a new point of attack within RLHF platforms, which, to the best of our knowledge, has not been previously studied.

3 Background & Preliminaries

Similar to most reinforcement learning problems, the RLHF alignment process, as a sequential decision-making problem, can be modeled as a form of Markov Decision Process (MDP). The MDP for RLHF can be represented as a tuple $(S, A, \mu, P, R, \gamma)$. S represents the state space, which comprises all previous text represented as a sequence of tokens. This includes the user’s initial prompt as well as previously generated tokens by the LLM, A represents the action space, which is the set of all tokens the LLM can generate next. μ is the distribution of all possible initial states. $P(s_{t+1} = s' \mid s_t = s, a_t = a)$ is the transition function, which represents the transmission to a new state s_{t+1} given the current state s_t and the taken action a_t . $R : S \times A \rightarrow \mathbb{R}$ represents the reward function which is not directly available. In a general form of MDP, discount factor $\gamma \in (0, 1]$ can be defined, however, in the context of RLHF alignment, due to the distinct nature of optimization in LLMs, defining a discount factor may not be useful (we assume $\gamma = 1$).

Suppose a trajectory of state-action pairs $\tau = \{s_0, a_0, s_1, a_1, \dots, s_{n-1}, a_{n-1}, s_n, a_n\}$ which represents what token is selected by the LLM as an action $a_t \in A$ given the current state $s_t \in S$ in each timestep t . Unlike the standard MDP, in the MDP for RLHF, a numerical reward signal $r = R(s, a)$ is not available directly with data collected from human feedback. In RLHF, the agent receives feedback from the environment in the form of preference data $\tau_i \succ \tau_j$, indicating that τ_i is preferred to τ_j according to the oracle reward (e.g. human evaluator). Thus, RLHF can be considered as a form of Preference-based Reinforcement Learning (PbRL). In PbRL, the policy is learned by directly comparing two state-action trajectories, with one being preferred over the other. This process involves modeling preferences from data in the form of pairwise comparisons, which is also known as preference learning (Fürnkranz & Hüllermeier, 2010; Wirth et al., 2017).

For simplicity, we adopt the notations used in (Chaudhari et al., 2024). Instead of considering a trajectory of state-action pairs τ , we represent the text provided by the user as *context*, denoted by c , and the response generated by the LLM as *output*, denoted by o . The oracular reward function is represented as $Pr(o_i \succ o_j \mid c)$, indicating the probability of response o_i is preferred over response o_j given context c .

In numerous RLHF methods, a utility learning approach is employed to leverage the learning algorithms designed for standard MDPs. In this approach, a utility function is trained to estimate a numerical value based on the human’s preferences. To this end, a reward model is trained to assign a numeric value to a text generated by an LLM, given a prompt, and can be mathematically shown as: $R : \mathcal{C} \times \mathcal{O} \rightarrow \mathbb{R}$, where $\mathcal{C}$ represents the set of all possible contexts and $\mathcal{O}$ represents the set of all possible responses. The reward model in RLHF is a language model fine-tuned on human preference data to assign a numeric score to a given prompt-response pair $r = R(c, o)$.

The most common strategies for training the reward model using human feedback include rating-based and preference-based approaches. Following previous works (Ouyang et al., 2022; Bai et al., 2022), we adopt preference-based feedback for its simplicity and effectiveness. In this approach, by collecting N ranking data samples, $N(N - 1)/2$ preference pairs are generated to train the reward model. However, providing feedback as preference data in a pairwise comparison cannot be directly used for RL training. Therefore, the reward model must learn a relationship between these pairwise comparison data and a scalar value. To learn the reward model from pairwise comparison data, Bradley-Terry-Luce (BTL) model (Bradley & Terry, 1952) is used, which defines a probabilistic model for the oracle reward as follows:

$$Pr(o \succ o'|c) = \frac{\exp(R(c, o))}{\exp(R(c, o')) + \exp(R(c, o))} \quad (1)$$

$$= \frac{1}{1 + \exp(R(c, o') - R(c, o))} \quad (2)$$

$$= \sigma[R(c, o) - R(c, o')] \quad (3)$$

where the $\sigma(x) = \frac{1}{1+e^{-x}}$ is sigmoid function and $R(c, o)$ and $R(c, o')$ are the rewards corresponding to output o and o' generated by LLM given the context c . The objective of RLHF is to improve the initial policy $\pi(a|s)$, which is a pre-trained language model, to better align with human preferences. To achieve this, we enhance the responses generated by the LLM while penalizing deviations that are significantly different from the original pre-trained model. This is done by leveraging Kullback-Leibler (KL) divergence, which measures the difference between the target policy $\pi(o|c)$ and the reference policy $\pi_{\text{ref}}(o_{\text{ref}}|c)$. Incorporating KL divergence, the final reward assigned to the generated response by LLM can be represented as:

$$R_\phi(c, o) = R(c, o) - \beta D_{\text{KL}}(\pi(o|c) || \pi_{\text{ref}}(o_{\text{ref}}|c)) \quad (4)$$

where $R_\phi(\cdot)$ returns the reward considering KL divergence, $D_{\text{KL}}(\cdot || \cdot)$ is KL divergence, and $\beta \geq 0$ is KL penalty. The objective function can be shown as follows:

$$J(\pi) := \mathbb{E}_\pi \left[\sum_t R_\phi(s_t, a_t) \right] = \mathbb{E}_\pi \left[\sum_t R_\phi(c, o_{1:t-1}, o_t) \right] \quad (5)$$

Since the numeric reward is an estimation of the actual oracular reward provided by the trained reward model at the end of the generated text, we can further simplify Equation 5 and represent the estimated objective function as follows:

$$\hat{J}(\pi) := \mathbb{E}_{c \sim d_C(\cdot)} [\mathbb{E}_{O \sim \pi(\cdot|c)} [R_\phi(c, O) | C = c]] \quad (6)$$

where $d_C(\cdot)$ represents the distribution of contexts in the training dataset.

During the RLHF alignment process, the agent aims to achieve the optimal policy (aligned language model) that maximizes the objective function as follows:

$$\pi^* = \arg \max \mathbb{E}_{c \sim d_C(\cdot)} [\mathbb{E}_{O \sim \pi(\cdot|c)} [R_\phi(c, O) | C = c]] \quad (7)$$

4 Misalignment in an Adversarial RLHF Platform

Our proposed attack on RLHF platforms can corrupt the alignment of LLMs by poisoning the reward model through manipulation of the preference dataset. In this strategy, the adversarial RLHF platform detects whether a user's task aligns with the attacker's objective. If it does, the platform manipulates the alignment process.

4.1 Task-specific Misalignment

In our proposed attack, the attacker integrates a classification model Θ to identify data samples related to their targeted topics. This model is pre-trained on a diverse dataset to detect instances associated with a specific concept that the attacker intends to either promote or demote. Upon detecting such samples within the preference dataset, the platform categorizes the user's task as a targeted task and identifies the relevant samples for manipulation. This classifier can be considered

as a function Θ that returns 1 if the input data sample is classified as related to the attacker’s targeted topic, and 0 otherwise, defined as:

$$\Theta(x) = \begin{cases} 1 & \text{if } x \text{ related to targeted topic(s),} \\ 0 & \text{otherwise.} \end{cases} \quad (8)$$

Let D_{pref} denote the preference data set sent to the RLHF platform by user. The integrated classifier within the platform identifies D_{target} which is a subset of D_{pref} related to the targeted topic, as follows:

$$D_{target} = \{x \in D_{pref} \mid \Theta(x) = 1\} \quad (9)$$

To corrupt the reward function and, consequently, the RLHF alignment, the attacker manipulates all or a portion of $D_{target} \in D_{pref}$ using label-flipping attack.

4.2 Label-flipping Attack

The preference dataset is typically in the form of pairwise comparisons, where each pair includes a chosen response o and a rejected response o' for a given prompt. This dataset captures human preferences, expressed as $o \succ o'$, and is used to train the reward model.

In the label-flipping attack, after identifying the targeted data samples D_{target} in the preference dataset D_{pref} , the labels of all or a portion of the data samples in D_{target} are reversed. Specifically, for each targeted sample, the label of the chosen response o is swapped with the rejected response o' . Thus, the comparison pair $o \succ o'$ is changed to $o \prec o'$. From this point forward, we denote the unattacked (clean) dataset as D and attacked dataset as D^- .

When the manipulated dataset D_{pref}^- is used to train the reward model, it learns the incorrect preferences for data samples related to a specific topic targeted by the attacker. As a result, the poisoned reward model $R^-(c, o)$ assigns inaccurate rewards (either higher or lower than the expected rewards) to these samples.

Without loss of generality, the final reward and objective functions remain as defined in Equations 4, 6 and 7, except that they now use the poisoned reward model R^- instead of the original one R . Consequently, this alteration leads to a different learned optimal policy π^{*-} , deviating from the one originally intended by the user π^* . This misalignment process is explained in Algorithm 1.

Algorithm 1 LLM Misalignment in an Adversarial RLHF Platform

Input: preference dataset D_{pref} , classifier Θ , sampling threshold n , base (pre-trained) reward model RM_{ref} , reward model learning hyperparameters θ_{RM} , base (pre-trained) LLM π_{ref} , RLHF fine-tuning hyperparameters θ_π

Output: maliciously aligned LLM denoted by π^{*-}

- 1: $D_{target} \leftarrow \{x \in D_{pref} \mid \Theta(x) = 1\}$ ▷ Identify targeted data samples
 - 2: $D'_{target} = \text{random_select}(D_{target}, n)$ ▷ randomly select n samples from D_{target}
 - 3: $D_{pref} \leftarrow D_{pref} - D'_{target}$
 - 4: **for** each data sample $x = (o, o') \in D'_{target}$ **do**
 - 5: Replace $x = (o \succ o')$ with $x^- = (o \prec o')$ ▷ flip the chosen and rejected labels
 - 6: **end for**
 - 7: $D_{pref}^- \leftarrow D_{pref} + D'_{target}$ ▷ D_{pref}^- contains manipulated samples from D'_{target}
 - 8: $RM^- \leftarrow RM_Training(D_{pref}^-, \theta_{RM}, RM_{ref})$ ▷ Train poisoned RM on dataset D_{pref}^-
 - 9: $\pi^{*-} \leftarrow RLHF(\pi_{ref}, RM^-, \theta_\pi)$ ▷ RLHF alignment using manipulated RM
-